BI2025 Experiment Report - Group 84

Jonathan Maier*
TU Wien
Austria

Fritz Marvin†
TU Wien
Austria

Abstract

This report documents the machine learning experiment for Group 84, following the CRISP-DM process model. The primary business objective was to develop a supportive tool for salary adaptation, specifically to assist in identifying existing employees who may be over- or under-compensated, as well as to benchmark appropriate salaries for new hires. Leveraging data from the Stack Overflow 2025 Annual Developer Survey, we formulated a linear regression task to predict developer salaries. While modern salary prediction often utilizes complex non-linear algorithms, we deliberately selected a Ridge Regression model for this experiment. Despite this untypical choice for high-dimensional survey data, our model achieved predictive results comparable to current state-of-the-art benchmarks established in previous years (e.g., on the 2024 dataset).

CCS Concepts

• **Computing methodologies** → **Machine learning**.

Keywords

CRISP-DM, Provenance, Knowledge Graph, Machine Learning, Linear Regression, Salaray Prediction, Stack Overflow Developer Survey 2025,

ACM Reference Format:
Jonathan Maier and Fritz Marvin. 2025. BI2025 Experiment Report - Group 84. In *Proceedings of Business Intelligence (BI 2025).* ACM, New York, NY, USA, 20 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Business Understanding

The Business Understanding phase serves as the foundational step of the CRISP-DM methodology. In this chapter, we outline the project's scope by defining the specific business problem, the available data, and the criteria for success as well as potential AI risk aspects.

1.1 Data Source and Scenario

We founded the company "Fritz & Maier AG" after our successfully completed Bachelors. It is a software company that has scaled rapidly to hundreds of employees located all over the world. Due to this rapid expansion, the current compensation structure is ad-hoc, resulting in internal pay inequities and potential misalignment with the current market. As the company matures and grows even further, HR requires a data-driven approach to standardize these salaries. The goal is to use the Stack Overflow data to analyze the relationship between skills, experience, and education against compensation to benchmark current employee salaries and make adaptations for the following business year. To achieve this, the analysis utilizes the Stack Overflow Annual Developer Survey 2025. This dataset contains comprehensive information regarding developer demographics, experience, technology usage, and compensation from a global audience.

1.2 Business Objectives

The initiation of this project is driven by the need to modernize the Human Resources compensation strategy. We have broken this down into one primary objective and three supporting objectives to ensure a comprehensive improvement in our salary processes.

1.2.1 Objective 1. The primary objective is to establish a data-driven salary benchmarking model to find a fair market value for software engineering roles. This model must calculate compensation based on objective factors such as professional experience, geographical location, and specific technology stacks, rather than subjective negotiation.

1.2.2 Objective 2. Our second objective focuses on internal equity. We aim to identify current employees who may be significantly under- or overpaid compared to the market. This identification allows HR to proactively address retention risks or budget inefficiencies.

1.2.3 Objective 3. The third objective is to standardize salary offers for new hires. By having a reliable model, we aim to reduce the time spent on negotiations and create a more transparent hiring process that instills confidence in candidates.

1.2.4 Objective 4. The fourth objective is budget optimization. We aim to ensure the company's financial resources are utilized efficiently by avoiding arbitrary overpayment while maintaining a competitive stance in the talent market.

1.3 Business Success Criteria

To evaluate whether the business objectives have been met, we have defined specific, measurable criteria for success.

1.3.1 Criterion 1. The first criterion addresses the adoption of the model. The project will be considered successful if the HR department adopts the model to review at least 80% of current employee contracts.

*Student A, Matr.Nr.: 12205610
†Student B, Matr.Nr.: 12129099

BI 2025, 17

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

1.3.2 Criterion 2. The second criterion focuses on retention. We aim to see a measurable reduction in staff turnover that is specifically attributed to compensation issues.

1.3.3 Criterion 3. The third criterion regards recruitment efficiency. Success is defined as an increase in the acceptance rate of initial job offers and a reduction in the time-to-hire, proving that our data-backed offers are competitive.

1.3.4 Criterion 4. The fourth criterion is financial accuracy. The project is successful if we achieve a total salary budget that aligns within +/- 10% of the predicted market benchmark, ensuring the company is neither overspending nor significantly underpaying.

1.4 Data mining goals

In order to exactly know what our goals are regarding the creation of this model, below now follow the data mining goals.

1.4.1 Goal 1. The general overall goal of our data mining activities is to build a regression model which helps HR determine appropriate salaries for new employees. And in order to do that we take the dataset provided by the Developer survey, prepare the data accordingly and build the model out of it.

1.4.2 Goal 2. Our second data mining goal is to identify employees that are significantly over- / underpaid in comparison to the market. Here we want to compare the predicted salary with the actual salary and afterwards be able to flatten out deviations.

1.4.3 Goal 3. Our third goal is to identify which skills or attributes / features influence the salary prediction of a developer the most. For this we want to look at the features separately and then identify the most driving ones.

1.4.4 Goal 4. The fourth and last here explicitly listed data mining goal is to analyze the relationship between salary deviations and indicators of job satisfaction, especially regarding underpaid developers, in order to identify the risk of an employee leaving the company early .

1.5 Data mining success criteria

To know, whether our model works as expected and has a certain standard of quality, below now follow clearly stated measurable success criterias.

1.5.1 Criteria 1. The first and most important data mining success criteria is to achieve a high accuracy regarding the salary predictions. Therefore we want to reach an R^2 of 80% minimum.

1.5.2 Criteria 2. Our second goal is to have a robust model which on average minimizes the predictions with large absolute errors. We consider the salary of an employee to be a large error, if he has at least +/- 3000 euros salary deviation of the initial model prediction. In this regard we use the MAE in order to measure this performance overall of our model.

1.5.3 Criteria 3. Our third goal is to have a high data quality for our model, which means, that after all data cleansing we have less than 10% of missing data across all features.

1.5.4 Criteria 4. Our fourth goal is to make the model readable for the HR department. As this is a subjective criteria it has to be checked whether the employees of the HR department are able to properly read the model and take conclusions out of it. In order to guarantee that, the creators of the model validate the usage of the HR department.

1.6 AI risk aspect

This section now discusses risks regarding our use case and dataset that could potentially have a negative impact throughout the lifecycle of our model.

1.6.1 Fairness & Ethics.

- The big advantage the chosen dataset has is that no differentiation between male and female developers has been made which means that the risk of the model building a bias in that regard is lower than it would have been with this feature.
- We also want to mitigate the risk of noisy or missing data as much as possible in order to be able to create a model that is as reliant as possible.
- In our opinion there are a few possible problems / biases the model could have, the most prominent one being the country that a developer is working in.

1.6.2 Data quality.

- Even though, the stack overflow survey consists of a broad range of developers that were asked, it does not represent the global developer population or their salaries.
- As the Fritz & Maier AG currently has their very own unique way of setting salaries, it may be possible, that the trained model strongly misaligns with the current salaries and therefore could be useless.

1.6.3 Documentation & Usage.

- Especially in fields like the salaries it is very important to have a high governance and documentation, in order to not only be compliant with the EU AI-Act, but also for the HR department to know, that the model can create biased predictions and therefore closely monitor / look at the predicted salaries.

2 Data Understanding

To address the business objectives of salary benchmarking and anomaly detection, this project utilizes data from the **Stack Overflow 2025 Annual Developer Survey**. As one of the most comprehensive snapshots of the software development industry, this dataset provides a diverse range of features including geographic location, developer experience, technology stack, and employment details.In this regard, this section documents the Data Understanding phase, detailing the initial collection, descriptive statistics, and exploratory analysis of the dataset. Furthermore, we assess data quality, focusing on missing values and potential biases that could impact the performance of our Ridge Regression model.

2.1 Attribute Types

Representing the survey's fifteenth year, it aggregates over 49,000 responses from 177 countries. The data includes responses to 62

questions regarding 314 different technologies, providing specific insights into the adoption of AI agent tools, Large Language Models (LLMs), and community platforms [10].

Complementing the primary response data, the dataset provides a metadata schema file. This file acts as a data dictionary, defining the question text and expected response format for variables in the main dataset.

The primary data file contains 172 columns and 49,191 rows. The schema file consists of 6 columns and 139 rows, categorizing the survey questions into four distinct types, shown in Figure 1:

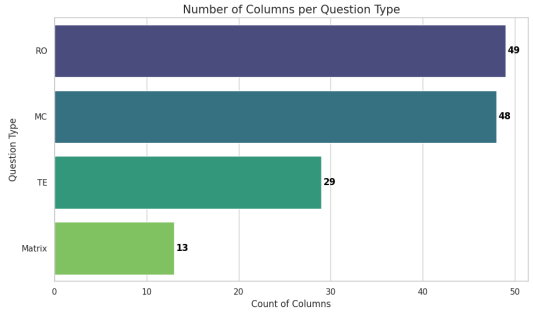


Figure 1: Listing of different question types with corresponding amount

From Figure 1 the following types can be extracted:

- **MC (Multiple Choice):** 48 columns
- **Matrix (grid-style questions):** 13 columns
- **RO (Rank/Read Options):** 49 columns
- **TE (Text Entry):** 29 columns

Ideally, each row in the schema corresponds to a column in the response data, providing context for the recorded values. However, as indicated by the count difference - 139 schema definitions versus the 172 data columns - the schema does not cover every variable present in the dataset. This discrepancy, including missing definitions and column name mismatches, is addressed in detail in the next Section 2.1.1.

2.1.1 Schema Consistency and Column Mismatch. To validate the structural integrity of the dataset, we performed a programmatic comparison between the column names in the primary dataset and the `qname` identifiers in the schema. This was executed using set operations to calculate the intersection and relative complements of the variable lists.

The automated consistency check yielded the following results:

- **Common Columns:** 126 variables matched perfectly between data and schema.
- **Data-Only Columns:** 46 variables existed in the dataset but were missing from the schema.
- **Schema-Only Columns:** 13 variables were defined in the schema but not present in the dataset.

Manual analysis was required to reconcile these differences. The investigation revealed that the mismatch is primarily due to the "flattening" of complex question types:

(1) **Matrix Question Expansion:** The 13 columns found only in the schema (e.g., `Language`, `Database`, `AIAgentImpact`) correspond to "Matrix" style questions. In the dataset, these are expanded into multiple columns representing specific aspects or Likert scale responses (e.g., `LanguageHaveWorkedWith`, `LanguageWantToWorkWith`, `AIAgentImpactStrongly agree`).
(2) **Derived Features:** Several columns found only in the data are calculated or system-generated fields rather than survey questions. Key examples include `ResponseId` (primary key) and `ConvertedCompYearly` (the target variable for salary prediction).

Due to this structural divergence, we proceeded with a manual mapping of features to ensure the expanded columns were correctly interpreted according to their parent definitions in the schema.

Further inspection confirmed that the columns missing from the schema are not errors, but rather contextual expansions of the base questions. The dataset utilizes a suffix-based naming convention to distinguish between different response contexts for the same technology or tool category. We identified three primary suffixes that map back to the root schema definitions:

- **-HaveWorkedWith:** Captures the respondent's current usage and experience (e.g., mapped to the parent `Language` question).
- **-WantToWorkWith:** Captures future interest or learning goals.
- **-Admired/Desired:** Captures sentiment regarding specific technologies.

Consequently, for the modeling phase, features such as `LanguageHaveWorkedWith` were treated as the primary representation of the `Language` variable defined in the schema.

2.2 Statistical Properties

Given the high dimensionality of the dataset (172 columns), an exhaustive exploration of every variable was deemed impractical for the scope of this experiment. Consequently, we performed a preliminary manual feature selection prior to the statistical analysis.

This scoping process was driven by domain reasoning, prioritizing variables with a logical theoretical link to compensation (e.g., *WorkExp*, *Country*, *LanguageHaveWorkedWith*). While we acknowledge that this heuristic approach introduces potential selection bias it allows for a more focused and interpretable analysis. The following exploration is therefore restricted to this subset of candidate features.

As the dataset contains several "Matrix" style questions where respondents rank or endorse specific technologies and statements (e.g., `TechEndorse`, `TechOppose`, `JobSatPoints`), these questions would expand into dozens of individual binary or ordinal features, meaning they would introduce significant noise and sparsity.

To optimize dimensionality while retaining potential predictive signals, we implemented a statistical filtering process:

(1) **Grouping:** We isolated the high-cardinality ranking groups: *TechEndorse*, *TechOppose*, and *JobSatPoints*.
(2) **Correlation Analysis:** We calculated the Pearson correlation coefficient between every individual option in these groups and the target variable (`ConvertedCompYearly`).

(3) **Top-3 Selection:** For each group, we retained only the three options with the highest absolute correlation, dropping the rest.

2.2.1 Results. This process reduced the ranking feature count from 45 columns down to 9. Notably, the correlations were generally weak (absolute values < 0.05), suggesting that individual technology opinions are minor factors in salary determination compared to hard skills or location.

The selected features included:

- **JobSatPoints:** Options 11, 4, and 6 (Max correlation: -0.0477).
- **TechOppose:** Options 3, 9, and 13.
- **TechEndorse:** Options 8, 6, and 4.

All other ranking columns were dropped from the analysis. This combined approach reduced the initial 172 columns to a final subset of **31 high-impact features** for the modeling phase, meaning a total of 141 columns were excluded from the analysis, falling into the following primary categories:

- **Low-Correlation Ranking Items:** To reduce dimensionality, 36 specific ranking options (12 from each of the 3 analyzed ranking categories) were dropped after showing weak correlation with the target variable.
- **Future Interest & Sentiment:** Columns related to technologies developers *want* to work with or *admire*.
- **Metadata & Identifiers:** System artifacts such as `ResponseId`, `SOVisitFreq`, and `Currency`, which have no relevance for income prediction.
- **Detailed AI Usage:** Granular questions regarding specific AI tools were excluded to focus on broader adoption metrics.

2.2.2 Retained Features. The 31 retained columns represent the most significant predictors of compensation:

- **Demographics & Experience:** *Age*, *EdLevel*, *WorkExp*, *YearsCode*, and *Country*.
- **Professional Context:** *DevType*, *OrgSize*, *RemoteWork*, and *Industry*.
- **Applied Tech Stack:** Variables indicating current professional usage, such as `LanguageHaveWorkedWith`.
- **Key Ranking Drivers:** The specific subset of ranking points (e.g., from *JobSatPoints*) identified as having the highest correlation with salary.
- **Target Variable:** `ConvertedCompYearly` (Annual Salary in USD).

Feature Statistics and Distribution Analysis for selected Features

We analyzed the descriptive statistics for the 31 retained features to understand the central tendencies and variability of the dataset.

2.2.3 Demographic and Professional Profile. The "typical" respondent in this dataset fits a specific profile:

- **Status:** 76% identify as professional developers (`MainBranch`) and are currently employed.
- **Education & Age:** The modal age bracket is 25-34 years, and the most common educational attainment is a Bachelor's degree.
- **Experience:** The average respondent has approximately 16.5 years of coding experience (`YearsCode`) and 13.4 years of professional work experience (`WorkExp`). Both distributions are right-skewed (skew $\approx$ 1.2–1.5), indicating a long tail of highly experienced veterans.
- **Role:** "Developer, full-stack" is the most common job type, and the United States is the most frequently represented country.

2.2.4 Target Variable Analysis: Salary. The target variable, `ConvertedCompYearly`, exhibits extreme distributional characteristics that will pose challenges for linear modeling:

- **Central Tendency:** The mean salary is \$101,761, significantly higher than the mode of \$69,609, suggesting a strong right skew.
- **Outliers:** The maximum recorded salary is \$50,000,000 (likely data entry errors or currency conversion anomalies).
- **Skewness:** The variable has a massive skewness of **75.5** and kurtosis of **7059**, confirming a non-normal distribution.

This implicates an extreme non-normality that necessitates a robust outlier handling and likely a logarithmic transformation in the preparation phase. However, there will be a more detailed outlier analysis in Section 2.5.

2.2.5 Technical Stack and Sentiment.

- **Technology Combinations:** The `...HaveWorkedWith` columns show extremely high cardinality (e.g., `LanguageHaveWorkedWith` has over 15,000 unique values). This confirms that these fields contain semi-colon separated combinations (e.g., HTML/CSS, JavaScript, ...") rather than single categories.
- **AI Adoption:** The majority of developers (`AISelect`) report using AI tools daily, with "Favorable" being the dominant sentiment (`AISent`).
- **Job Satisfaction:** The overall job satisfaction (`JobSat`) is relatively high, with a mean of 7.2/10 and a negative skew (-1.01), indicating most developers are generally satisfied.

Correlation Analysis

To rigorously assess the linear relationships between the selected numerical features and the target variable, we computed a Pearson correlation matrix. The results were visualized in a lower-triangle heatmap (see Figure 2), where the color intensity represents the strength of the relationship - red indicating positive correlation and blue indicating negative correlation.

2.2.6 Methodology. The analysis was driven by a programmatic "Smart Drop" logic designed to identify two specific issues:

- **Weak Signals:** Features with an absolute correlation to the target of less than 0.05 ($|r| < 0.05$) were flagged for potential removal, as they contribute minimal predictive value to a linear model.
- **Multicollinearity:** Feature pairs with a correlation exceeding 0.60 ($|r| > 0.60$) were flagged. In standard regression pipelines, highly collinear variables inflate variance. The algorithm was designed to retain only the feature in the pair with the stronger relationship to the target.

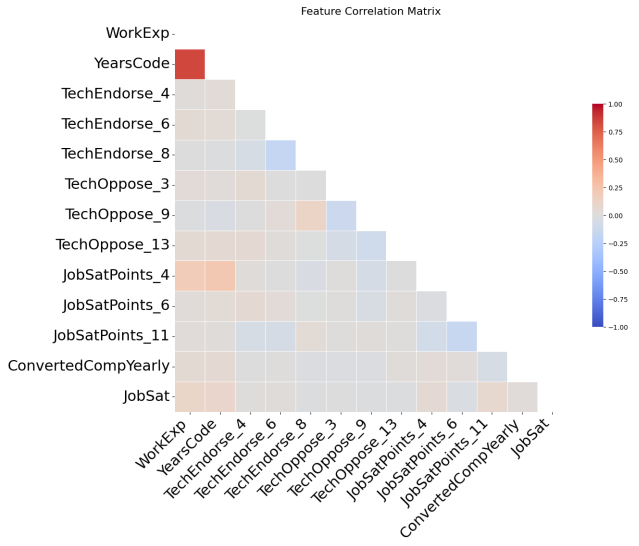


Figure 2: Pearson Correlation Matrix of selected numerical features. The deep red block highlights the strong multicollinearity between experience-related variables, while the faint bottom row indicates weak linear correlations with the target variable.

2.2.7 Visual Analysis and Results. The heatmap reveals a predominantly neutral field, indicating that most independent variables have low linear correlation with each other. This independence is generally favorable for regression models. However, two specific areas warrant closer inspection:

1. Feature-to-Feature Correlations (Multicollinearity). The most prominent visual feature is the deep red block in the top-left corner, representing the relationship between `YearsCode` and `WorkExp`. As expected, these variables are strongly positively correlated, exceeding the 0.60 collinearity threshold.

There is an excpetion to the rule, as although our algorithm suggested removing one of these variables to reduce redundancy, empirical testing during the modeling phase revealed that retaining both yielded superior performance. This suggests that while they overlap, they capture distinct nuances. `YearsCode` accounts for total coding history (including education and hobbies), while `WorkExp` strictly measures professional tenure. Therefore, we overrode the automated suggestion and retained both features.

2. Correlations with the Target (ConvertedCompYearly). The row corresponding to the target variable (`ConvertedCompYearly`) is notably faint across the board, with most cells showing near-zero color intensity. This lack of strong linear correlation highlights a critical data characteristic: the relationship between developer attributes and salary is likely non-linear, or heavily obscured by the extreme outliers identified in the previous statistics section (e.g., salaries of $50M). This visual confirmation supports the necessity for the non-linear transformations (such as log-scaling) applied in the subsequent Data Preparation phase.

2.3 Data Quality Assessment

To determine the dataset's fitness for the regression task, we performed a comprehensive quality audit focusing on missingness and data plausibility.

2.3.1 Missing Values Analysis. The dataset exhibits significant sparsity across several feature categories. Figure 3 illustrates the frequency of missing entries per column.

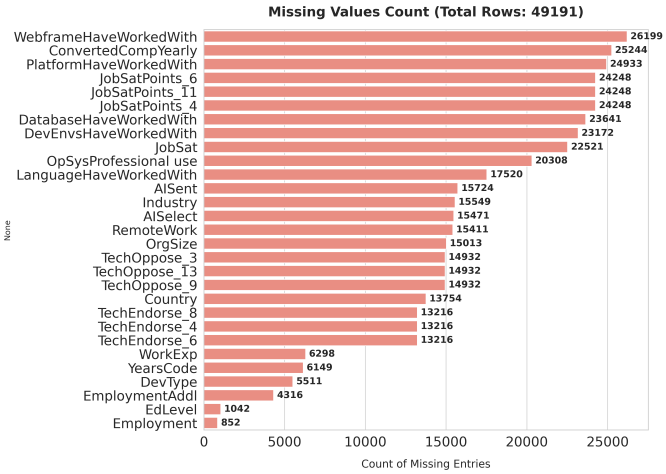


Figure 3: Count of missing entries per feature (Total Rows: 49,191). The target variable and detailed tech stack questions show high missingness.

After a detailed analysis of 3, these critical findings were identified:

- **Target Variable (`ConvertedCompYearly`):** The most severe quality issue is the absence of salary data in 25,244 records (approx. 51% of the dataset). Since supervised learning requires labeled data, these rows must be dropped during the preparation phase.
- **Tech Stack & Rankings:** High missingness in columns like `WebframeHaveWorkedWith` (26,199 missing) and `JobSatPoints` (24,248 missing) likely indicates "Not Applicable" or survey fatigue rather than random error. These will require imputation (e.g., filling with "None" or median values) rather than row deletion.
- **Demographics:** Core fields like `EdLevel` and `Employment` have very low missingness (< 2%), providing a stable base for the analysis.

2.3.2 Plausibility. Beyond missing data, we validated the logical consistency of the existing values. We visualize the distribution of key numerical variables against defined logic thresholds in Figures 4, 5, and 6.

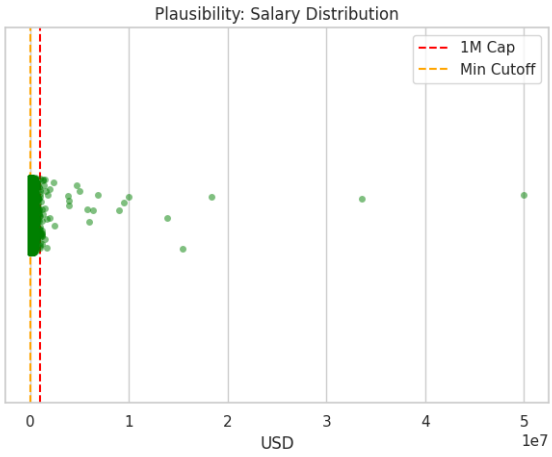


Figure 4: Plausibility Check: Salary distribution and identified outliers.

As illustrated in Figure 4, the salary distribution contains extreme outliers that would severely distort a linear model such as Ridge Regression. The strip plot reveals 46 records with salaries exceeding $1,000,000 (peaking at $50M) and 304 records below $100. These anomalies likely represent data entry errors or currency conversion issues.

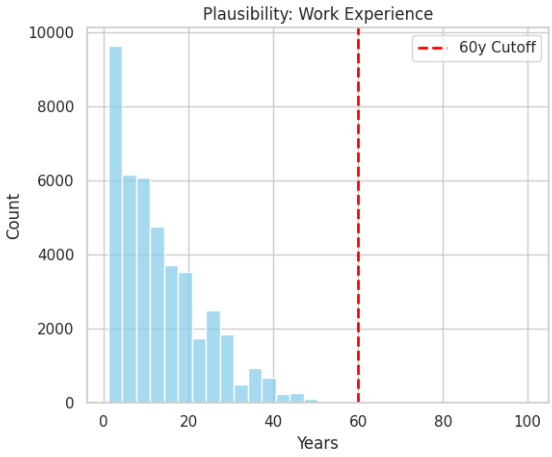


Figure 5: Plausibility Check: Work Experience distribution.

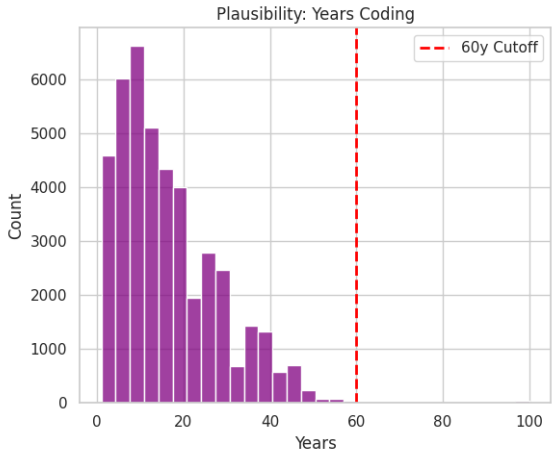


Figure 6: Plausibility Check: Years Coding distribution.

While rare, we detected 54 respondents claiming more than 60 years of work experience (see Figure 5). Similar to work experience, values exceeding 60 years for coding experience are flagged as implausible, with Figure 6 highlighting 71 such cases. The dashed lines in these figures mark the cutoff for valid tenure. Given the target demographic of the survey, these values are statistically suspicious and will be treated as noise.

This leads to the conclusion that the data quality assessment necessitates a rigorous cleaning pipeline, including dropping rows with missing targets, capping or trimming outliers, and imputing missing categorical features.

2.4 Visual Inspection Findings

While a comprehensive visual inspection was conducted for all 31 selected features, this section highlights only the most statistically significant patterns and structural anomalies that necessitate specific handling in the subsequent modeling phases. While many of these findings were already identified through numerical and statistical analysis, they are reiterated here for completeness, as visual inspection allows us to verify the specific nature and severity of these anomalies which summary statistics alone cannot fully capture.

2.4.1 Target Variable Distribution. Visual analysis of the `ConvertedCompYearly` feature confirms the extreme right-skewness identified in the statistical summary. The distribution is characterized by a massive concentration of values near the median, with a long tail extending to $50M. Unlike a standard normal distribution, the presence of these extreme outliers indicates that salary data in its raw form violates the homoscedasticity assumptions required for linear regression.

2.4.2 Systemic Multi-Label Complexity. A defining characteristic of the technical stack features (e.g., `Language`, `Platform`, `Webframe`) is their combinatorial nature. Visual inspection reveals that these columns are not single-category factors but semi-colon separated strings (e.g., "Next.js;Node.js;React"). This structure confirms that

developers rarely use tools in isolation, and the frequency distributions are dominated by complex combinations rather than individual technologies.

2.4.3 Demographic and Professional Imbalance. The dataset exhibits a strong selection bias towards established professionals. The `MainBranch` category "I am a developer by profession" vastly outweighs other groups such as students or hobbyists. Similarly, the `Industry` feature is heavily skewed towards "Software Development," with other sectors like Fintech or Healthcare representing a much smaller fraction of the sample.

2.4.4 Work Environment Trends. The visual data regarding work modes offers a clear snapshot of the post-pandemic landscape: "Remote" and "Hybrid" work arrangements have become the statistical norm, while "In-person" work is a distinct minority. Additionally, the `OpSys` feature highlights a high prevalence of "Windows Subsystem for Linux (WSL)," suggesting that operating system usage is no longer binary (Windows vs. Linux) but increasingly hybrid.

2.4.5 Ordinal and Sentiment Structures. The inspection of ordinal features revealed two distinct distribution types:
Organizational Size, which displays a clear ordinal hierarchy (from "Just me" to "10,000+ employees") which is currently encoded as strings. **Sentiment Scores**, which are Features such as `JobSatPoints`, `TechEndorse`, and `TechOppose` display discrete, "spiky" distributions at integer intervals. `JobSat` specifically is left-skewed (peaking at 7-8), indicating that the sample population is generally satisfied with their roles.

2.4.6 Experience vs. Salary Outliers. Comparing the distributions of `WorkExp` and `ConvertedCompYearly` reveals a critical distinction in outlier types. While both are right-skewed, the long tail in the experience data (e.g., developers with 40+ years of tenure) appears statistically plausible and represents a "veteran" cohort. In contrast, the extreme outliers in the salary data are likely artifacts or errors that do not reflect valid market variations.

2.5 Outlier Analysis and Handling Strategy

We performed a targeted analysis of the numerical features, specifically the target variable (`ConvertedCompYearly`) and experience metrics (`WorkExp`, `YearsCode`), to identify high-leverage points that could destabilize the linear regression model.

2.5.1 Visual Inspection of Distributions. As already found in Section 2.4 and illustrated in Figure 7, the target variable is characterized by extreme right-skewness. The original distribution (left panel) is dominated by a tight cluster near zero and a long tail extending to $50M, rendering the boxplot interquartile range (IQR) nearly invisible.

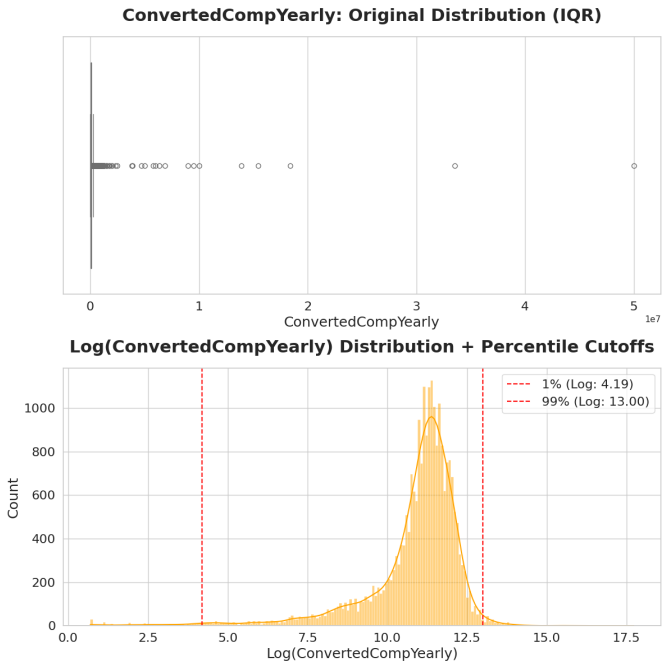
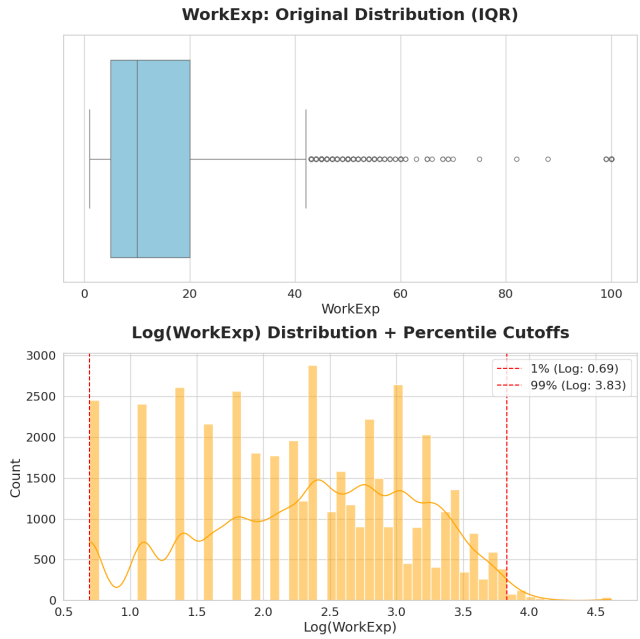
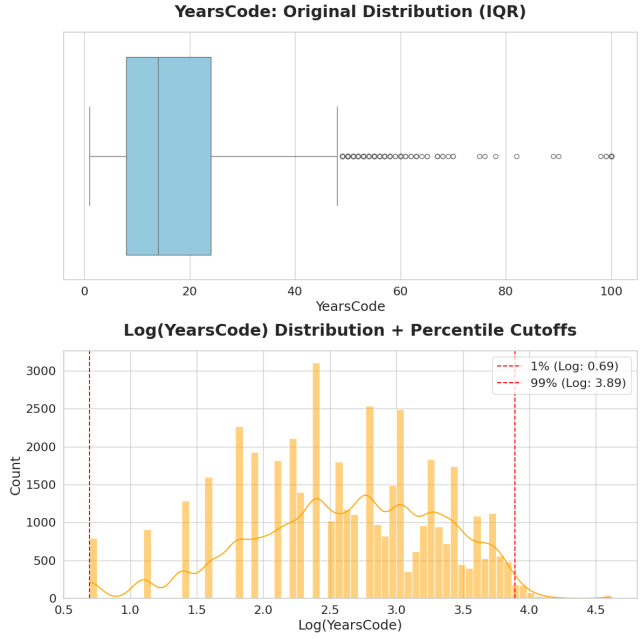


Figure 7: Target Variable Analysis. Left: Original boxplot shows extreme compression due to outliers up to $50M. Right: Log-transformed distribution reveals clearer bell curve, with red lines indicating proposed 1% and 99% capping thresholds.

Similarly, the experience variables (Figures 8a and 8b) reveal a "veteran tail." While physically possible, respondents with > 50 years of experience act as statistical outliers in a linear context.



(a) Work Experience



(b) Years Coding

Figure 8: Experience Distributions. Standard IQR method (boxplots) aggressively flags senior professionals (30+ years) as outliers. Log-transformed histograms suggest capping at 99th percentile is a more data-conserving strategy.

2.5.2 Methodological Decision: IQR vs. Percentile Capping. Initial tests using the standard Interquartile Range (IQR) method for outlier removal proved problematic. As seen in the boxplots above, the IQR method classifies a significant portion of valid data (e.g., senior developers with 40+ years of experience) as outliers. Removing these entries would bias the model against high-experience professionals, which lead to the selection of Log-Transformation in combination with 99th Percentile Capping.

In the following methods to address these outlier issues will be discussed in detail, which are then summarized within the Action Plan, described in Section 2.8. To balance regression stability with data preservation, we adopted the following strategy for the Data Preparation phase:

(1) **Logarithmic Transformation:** We will apply a log-transform ($log(1+x)$) to `ConvertedCompYearly`, `WorkExp`, and `YearsCode`. This naturally compresses the right skew, pulling extreme values closer to the mean without discarding them.
(2) **Percentile Capping:** instead of the strict IQR rule, we will clip values at the **1st and 99th percentiles**. This removes only the truly implausible extremes (e.g., salaries < \$100 or > \$1M) while retaining valid high-value data points.

2.5.3 Categorical and Ordinal Feature Analysis. Beyond numerical extremes, we analyzed the distribution of categorical and ordinal variables to identify structural imbalances:

- **Categorical Long Tails:** Features such as `DevType`, `Country`, and `Industry` exhibit high cardinality with long tails of rare labels. To prevent model overfitting on sparse categories, we will apply **Frequency Thresholding** in the Data Preparation phase, grouping rare labels into a generic "Other" category. Additionally, `MainBranch` shows extreme imbalance and will be handled via binary grouping (e.g., "Professional" vs. "Non-Professional").
- **Ordinal Validity:** Analysis of `JobSat` confirmed that low values (0-2) represent valid negative sentiment rather than errors; these will be retained. Conversely, non-informative labels in other ordinal columns (e.g., "I don't know" in `OrgSize`, "Prefer not to say" in `Age`) do not reflect scalar rank and will be flagged for exclusion or imputation.
- **Ranked Features:** As noted in the Feature Selection section, the ranking columns (e.g., `TechEndorse`) have already been filtered by correlation. The remaining features are deemed statistically significant and require no further outlier treatment.

2.6 Ethical Aspects

A significant ethical consideration regarding this model is the suitability of broad survey data for predicting individual compensation. Survey data relies on general statistics and may fail to account for an individual's unique context. Utilizing this model for real-world compensation decisions carries the risk of unfairly capping salaries for employees who do not align with the "average" profile, potentially negatively impacting livelihoods.

Furthermore, the data exhibits potential geographic and economic bias. Respondents are clustered into distinct economic zones—for example, those in Ukraine (reporting in UAH) versus Western nations (reporting in USD or EUR). If the model fails to account for cost-of-living disparities, it may unfairly categorize senior developers from certain regions as "low value." Additionally, ageism is

a concern; the data aggregates all respondents over the age of 65 into a single category, which may lead the model to underestimate the technological proficiency of older demographics. Finally, a systemic preference for formal education over self-taught skills risks penalizing experienced developers who lack traditional university credentials.

2.7 Risks and Bias

A fundamental risk to the analysis is the veracity of self-reported data. Survey responses may contain exaggerated claims or inaccuracies. If this response bias correlates with specific locations or age groups, it introduces hidden latent biases that are difficult to detect.

Financially, the `CompTotal` variable presents significant data quality risks due to mixed currencies. Respondents report figures ranging from thousands (EUR) to millions (UAH) without standardization. Due to this lack of comparability, `CompTotal` is excluded from the final dataset. The analysis will instead rely on `ConvertedCompYearly`. However, this variable contains missing values for a significant portion of the user base (e.g., students), resulting in data gaps. Subjective features also pose a risk; for instance, a respondent's sentiment regarding AI could skew self-reported productivity metrics.

2.8 Action Plan

The data preparation strategy prioritizes validity for regression analysis. The process begins by removing rows where the target variable `ConvertedCompYearly` is missing, as imputing salary would introduce unacceptable noise relative to the business objective. To prevent multicollinearity and magnitude errors, the `CompTotal` and `Currency` columns will be removed. In the following the recommendations for the data pre-processing step are clearly defined:

(1) **Target Cleaning:** Rows with missing values in the target variable `ConvertedCompYearly` are removed to ensure valid regression targets.
(2) **Outlier Handling (Log & Percentile):** Instead of the standard IQR method, we apply a log-transformation followed by the 99^{th} percentile method to `ConvertedCompYearly`, `WorkExp`, and `YearsCode`. This approach effectively manages the extreme skewness and high-leverage points (e.g., > 50 years experience) without discarding valid veteran data.
(3) **Imputation:** Missing values in `WorkExp` are imputed using the median experience of corresponding age groups. Missing values in `TechEndorse` are assumed to indicate no endorsement and are filled with 0.
(4) **Ranking Inversion:** For columns belonging to a ranking group (e.g., `TechEndorse_1`), the ordering is inconsistent with a "higher is better" magnitude. We inverse the order of these values so that higher numbers represent stronger endorsement.
(5) **Feature Encoding:**
- **Multi-Select:** Semicolon-delimited strings in `LanguageHaveWorkedWith` are split into separate boolean columns (multi-hot encoding).
- **Ordinal:** Ranges in columns like `OrgSize` are mapped to numeric points preserving the order.
- **Nominal (Top-N):** Cardinality for `Country` is reduced by keeping the top N countries and binning the rest into "Others". Similar frequency thresholding is applied to `DevType` and `Industry`.

(6) **Feature Selection & Scaling:** Columns containing personal decisions offering no utility (e.g., "Prefer not to say" in `Age`) are flagged for removal. Finally, a Scaler is applied to all numeric features ensuring that features with larger magnitudes do not disproportionately influence the Linear Regression weights.

3 Data Preparation

This section details the transformation pipeline applied to the raw survey data, covering missing value imputation, outlier management, and feature encoding to ensure a robust dataset for regression modeling.

3.1 Missing values

Rows with missing values in the target variable ConvertedCompYearly were removed, since a valid salary label is required for our regression model and such records cannot be used for training or evaluation. Further, using mean values was also not the way to go for us as it creates untrue / artificial values. For the two key experience-related predictors, WorkExp and YearsCode, missing values were imputed using the median of each column. Median imputation was chosen because experience-related values can vary strongly and sometimes contain very large values, which would distort the mean.

3.2 Handle Outliers

Regarding outliers, after the Data Understanding phase we thought it would be to good remove outliers of the same three features that were processed regarding the missing values as well: ConvertedCompYearly, WorkExp and YearsCode. The outliers can be seen in the graphics below:

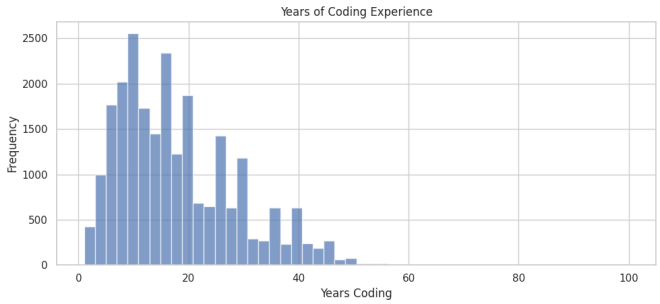


Figure 9: Before removal of outliers 2

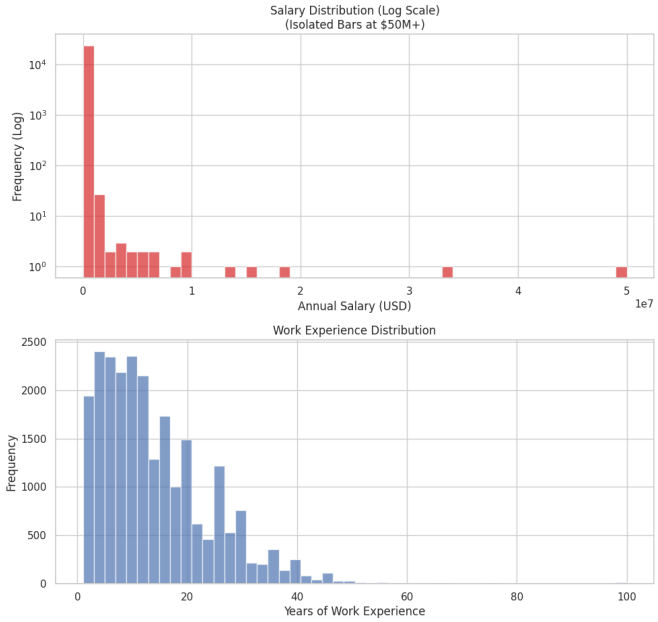


Figure 10: Before removal of outliers 1

Initially, everything above the 99th percentile was removed. Afterwards, during the Evaluation phase I also played around and after some iterations I found out, that going lower than the 95th percentile does not significantly improve the model anymore. I did the same iterating steps for WorkExp and YearsCode where the values below the 1st and above the 99th percentile should be removed. Below are the values after the removal:

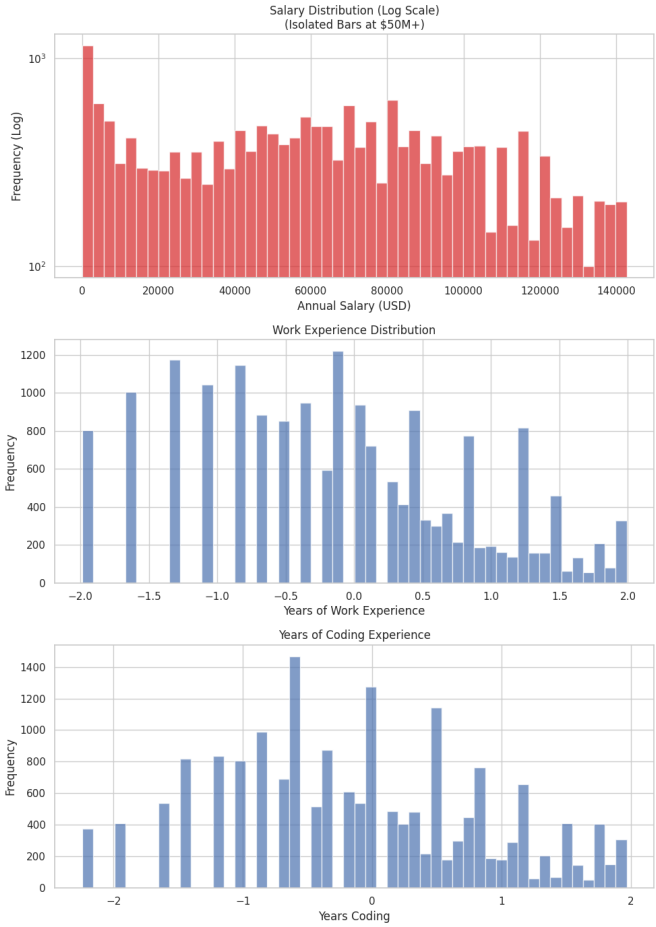


Figure 11: After removal of outliers

3.3 Inverting Ranking

Initially there have been a lot of columns regarding TechEndorse and JobSatPoints. The within the Data Understanding phase most important one have been kept, the other ones deleted. Note: The * at the end is a placeholder for different numbers.

- TechEndors_*
- JobSatPoints_*
- JobSat

The problem is, that giving a 1 here was meant that it is the best possible ranking a developer could give. But as our regressions model should take higher values as "more" or "better" it was clear that it is necessary to invert these features.

3.4 Ordinal encoding

A lot of questions asked by the survey are represented as different categories, which are then not represented as numeric values within the dataset set. The affected columns were the following:

- Age
  - Under 18 years old
  - 18-24 years old

  - 25-34 years old
  - 35-44 years old
  - 45-54 years old
  - 55-64 years old
  - 65 years or older
  - Prefer not to say
- Education
  - Primary/elementary school
  - Secondary school (e.g., American high school, German Realschule or Gymnasium, etc.)
  - Some college/university study without earning a degree
  - Associate degree (A.A., A.S., etc.)
  - Bachelor's degree (B.A., B.S., B.Eng., etc.)
  - Master's degree (M.A., M.S., M.Eng., MBA, etc.)
  - Professional degree (JD, MD, Ph.D, Ed.D, etc.)
  - Other
- Organizaion size
  - Just me – I am a freelancer, sole proprietor, etc.
  - Less than 20 employees
  - 20 to 99 employees
  - 100 to 499 employees
  - 500 to 999 employees
  - 1,000 to 4,999 employees
  - 5,000 to 9,999 employees
  - 10,000 or more employees
  - I don't know
- AI usage
  - Yes, I use AI tools daily
  - Yes, I use AI tools weekly
  - Yes, I use AI tools monthly or infrequently
  - No, but I plan to soon
  - No, and I don't plan to
- Feelings towards AI
  - Very favorable
  - Favorable
  - Indifferent
  - Unfavorable
  - Very unfavorable
  - Unsure

Within each feature each category then got an individual number assigned, where a higher number was "more" or "better" again. A Bachelor degree for example had a higher number than the primary/elementary school.

3.5 Multi Select Columns transform

Some questions also allowed multiple answers to be given, which then resulted in a string being displayed in the dataset, for example regarding the programming languages a developer has worked with have been displayed like "Python","Java","C++". Overall, the following columns were affected:

- Programming Languages
- Databases
- Cloud Plattforms
- Web Frameworks
- Development Environments
- Operating Systems
- Additional Employments

To solve the problem I used the 1-to-n encoding which is shown in the lecture slides as well. For every answer possibility of every feature an own column was created which was filled with a 1 if checked with a 0 if not.

3.6 Categorical columns transform

Similar to the Multi-Select columns some form of 1-to-n encoding was used regarding the Categorical features as well. Plain 1-to-n encoding was used for the following features:

- Main Branches (where a developer works in)
- Employment status
- Development Types
- Industries
- Remote Work

Regarding the Development types it is important to note, that we considered some of the types not relevant for our model and therefore deleted these rows. The decision was completely subjective made by student b regarding our use case.

Regarding the country a developer works in Top-N encoding was used, as using all possible countries as an own column would have blown up the dataset to much. Top 19 countries were chosen in order to their own column and the rest of the Countries was summarized within an "Other" column.

3.7 Removal of not needed columns

This step initially deleted all columns which were processed and not needed anymore like the ordinal and categorical feature columns. Later on when it came to the modelling part of student a we realized that some columns which already should have deleted by student a above have not been deleted, I just added the affected columns in this part as well in order to make sure that they are always properly deleted when going through the notebook.

3.8 Scaling

After the initial modeling and evaluation part we didn't have this part of the data preparation, but after my team partner and I had a little chat we figured, that we missed out on scaling certain columns and that implementing it would help us in reach better results with our model. So, after again playing around with scaling algorithms the MinMaxScaler was used for the numerical columns where the value was inverted and for the ordinal mapped values. Further, the continuous variables WorkExp and YearsCode were transformed using a Yeo–Johnson power transformation.[3]

3.9 Preprocessing steps considered but not applied

This section now covers possible preprocessing steps that my colleague and I discussed, but in the end never applied.

3.9.1 Removing Outliers. Regarding the removal of outliers we had two separate approaches in order to handle them, one being the removal of every row that has a salary above 500.000 per year and the second one being the removal with the usage of perceptiles.

In the end we decided against taking absolute number as a border for removal, because it is in our opinion not wise to do that. Having absolute numbers as a salary cap me be working for now, but in the future when salaries generally rise, the border of 500.000 might be outdated and therefore could lead us to wrong decisions. With using perceptiles on the other hand, you always go with the relative flow of the numbers.

3.9.2 Binning. We did not use binning at all (apart of the already "binned" columns like age) because with binning for example the salaries of the developers we would have created classes which afterwards would have to be encoded again in order to be usable for our regression problem. And as this would have been a lot of work for no real benefit we did not do that.

3.9.3 Attribute removal. Regarding attribute removal we had a long discussion about which attributes to use and which to remove from our dataset, always debating whether these attributes would have contributed something to our problem / model or not. Especially regarding the attribute "RemoteWork", which basically listed all types of working (from fully remote to fully in the office) we had a long discussion about whether this attribute would be interesting or not. After a bit of searching through the internet we then decided to keep the attribute and therefore did not apply the step of attribute removal.

3.9.4 One-Hot Encoding for countries. We decided against doing one-Hot encoding for every country as it would have resulted in more than 150 columns to cover each country. We therefore decided to analyze how many countries developers are withing the Top 5, 10, 15, and 20 and then decided to go with Top-N-Encoding.

3.9.5 Textual fields. We considered to go through and analyze all columns which consisted of textual answers that were given by the developers, but decided against it, as it would have been far to much to analyze and also difficult to properly encode it with the right values. Therefore we decided to drop all these attributes which resulted in having less noisy features within our dataset.

3.10 Derived attributes

This section now focuses on attributes that could potentially be derived, but in the end wasn't applied.

3.10.1 Countries –> Regions. One possible option for making a derived attributes would have been to group countries to regions / continents. But as there are a lot of differences betweend for example western and eastern europe, we decided against it.

3.10.2 Work experience in years –> seniority level. We could have created a column and classify developers based on their working experience as junioer, senior, etc. But as the Working experience is a numerical input anyway and also is "better" the higher the value is, it didn't make much sense for us. We further think, that with kind of grouping developers based on their work experience, we might create a bias which we do not want.

3.10.3 Job satisfactory. Regarding the job satisfactory we also could have created derived attributes that for example say if you gave your job between 10-14 points yout job is satisfying, but with our regression problem we would have needed to encode it again as not satisfying for example would have gotten a 1 and satisfying a 2. This therefore wouldn't have made any sense as well.

3.10.4 1-to-n coding. The only real thing we did regarding the deriving of attributes was to make use of the 1-to-n encoding. As we had a lot of attributes that were either ordinal, Multi-Select or categorical, this was in our opinion the best way in order to transform the values of the affected attributes to make our dataset ready for the modeling.

3.11 External Data Sources

Below now follow three potential datasets that we could have used in order to train, validate and test our model. **22700+ Software Professional Salary Dataset**

- https://www.kaggle.com/datasets/whenamancodes/software-professional-salary-dataset

This could have been a good alternative to our chosen dataset, but this one only has 14 attributes and in the specification it would have been at least 15, so we wouldnt have been able to use this dataset anyways. Furthermore, this dataset does not have any attributes, that our chosen dataset wouldnt have anyway.

Developer_Salary_By_Country

- https://www.back4app.com/database/back4app/developer-salary-by-country/api

While doing our research we stumbled across this dataset right here, but in our opinion we wouldn't be able to derive some high quality model from this dataset.

Remote Software Developer Salaries Around the World

- https://arc.dev/salaries

If we wanted to dive deeper into the salaries regarding developers that are working remote, this dataset would have actually been interesting for us. But like the other two datasets, we did not use them.

Best Salary Datasets & Databases

- https://datarade.ai/data-categories/salary-data/datasets

This datasets also look very good on the first sight, but we would have to be very careful on using one of these datasets, because some of them only cover one country like USA. This would have created a population bias which we do not want.

3.12 Data Cleaning

Describe your Data preparation steps here and include respective graph data.

4 Modeling

This chapter details the construction, validation, and selection of the predictive model. Building upon the data preparation phase, we focus on establishing a robust regression baseline that balances predictive accuracy with interpretability. The following sections outline the algorithm selection, hyperparameter tuning strategy, and final performance evaluation.

4.1 Algorithm Selection

Consistent with the objectives defined in the Data Understanding phase, we aimed for a linear approach to establish a solid baseline. We selected **Ridge Regression (L2 Regularization)** as the primary modeling technique. This and the following section thereby reference the Ridge Regression Model as defined by the sklearn library in python[11].

4.1.1 Justification. Our dataset is characterized by a moderate row count ($N \approx 25,000$) but high dimensionality ($D \approx 400$ features) resulting from the extensive one-hot encoding of multi-select and categorical variables.

Standard Ordinary Least Squares (OLS) regression is highly vulnerable to overfitting in this setting due to multicollinearity (high correlati

4.2 Algorithm Selection

Consistent with the objectives defined in the Data Understanding phase, we aimed for a linear approach to establish a solid baseline. We selected **Ridge Regression (L2 Regularization)** as the primary modeling technique. This and the following section thereby reference the Ridge Regression Model as defined by the sklearn library in python[11].

4.2.1 Justification. Our dataset is characterized by a moderate row count ($N \approx 25,000$) but high dimensionality ($D \approx 400$ features) resulting from the extensive one-hot encoding of multi-select and categorical variables.

Standard Ordinary Least Squares (OLS) regression is highly vulnerable to overfitting in this setting due to multicollinearity (high correlation between features). McDonald highlights that in such "ill-conditioned" problems, OLS coefficient estimates become unstable and exhibit high variance, making them unreliable for prediction [4, 7]. Ridge Regression addresses this by adding an L2 penalty to the loss function:

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \sum \beta_j^2$$

This penalty shrinks coefficients (β) towards zero, effectively trading a small amount of bias for a significant reduction in variance [7], which prevents overfitting without eliminating variables entirely.

4.2.2 Alternative Algorithms Considered.

- **Lasso Regression (L1):** While useful for feature selection, Lasso can be unstable when features are highly correlated, as it tends to arbitrarily select one feature while reducing others to zero. Ridge is generally more robust in preserving signal from correlated groups [8].
- **ElasticNet:** This combines L1 and L2 penalties. While a valid alternative, it introduces an additional hyperparameter (`l1_ratio`) to tune, increasing computational complexity for this baseline phase [6].
- **Histogram-based Gradient Boosting (HGBC):** While HGBC is highly efficient for tabular data and handles nonlinear relationships better than Ridge, we prioritized Ridge to establish a high-interpretability baseline where feature importance (coefficients) can be directly analyzed [5].

4.3 Hyperparameter Selection

4.3.1 Tuning Strategy: Regularization Strength (α). The primary hyperparameter selected for optimization is α (alpha). In Ridge Regression, this parameter controls the regularization strength, governing the trade-off between model complexity and error.

- **Justification:** A value of $\alpha = 0$ is equivalent to standard OLS (high variance), while very high α values lead to underfitting (high bias). Given the high dimensionality of the dataset, optimizing α is critical to effectively dampen noise without losing predictive signal. We employ a logarithmic grid search to identify the optimal magnitude.

4.3.2 Fixed Hyperparameters. The following parameters were held constant to isolate the effect of regularization and maintain computational efficiency:

(1) **Solver (`solver='auto'`):** Scikit-learn's automatic selection logic was retained, as it efficiently chooses the optimizer (e.g., Cholesky vs. SAGA) based on dataset dimensions.
(2) **Tolerance (`tol=1e-4`):** The default convergence tolerance provides sufficient precision for this regression task; tuning offers diminishing returns compared to regularization adjustment.
(3) **Intercept (`fit_intercept=True`):** Essential for uncentered salary data (which has a non-zero mean).
(4) **Constraints (`positive=False`):** We explicitly allow negative coefficients, as certain features (e.g., junior roles or lower cost-of-living regions) may legitimately exert a negative influence on the predicted salary.
(5) **Reproducibility (`random_state=42`):** Fixed to ensure consistent results across stochastic solvers.
(6) **Memory (`copy_X=True`):** Kept at default to ensure input data is not overwritten during processing.

4.3.3 Final Configuration. The complete configuration used for the training phase is summarized in the table 1below:

Table 1: Hyperparameter Configuration for Ridge Regression

Parameter	Description	Strategy/Value
`alpha`	Regularization strength (L2 penalty)	**Tuned** (Log Grid)
`solver`	Optimization algorithm	`'auto'`
`fit_intercept`	Calculates model intercept	`True`
`tol`	Convergence tolerance	1×10^{-4}
`positive`	Force positive coefficients	`False`
`random_state`	Seed for reproducibility	42

4.4 Splitting Strategy

We employed a 70% Training, 15% Validation, and 15% Testing split. A random state of 42 was used to ensure reproducibility. Since this is a regression task on continuous salary data, standard stratification was not applied, but distribution checks (mean/std) confirm that the splits are statistically representative of the overall population.

4.4.1 Target Variable Consistency. To ensure the model is evaluated on data that closely resembles the training conditions, we visualized the distribution of the target variable (`ConvertedCompYearly`) across all splits.

Figure 12 displays the Kernel Density Estimate (KDE), showing that the training, validation, and testing curves overlap almost perfectly. This confirms that no specific salary range is over- or under-represented in the hold-out sets.

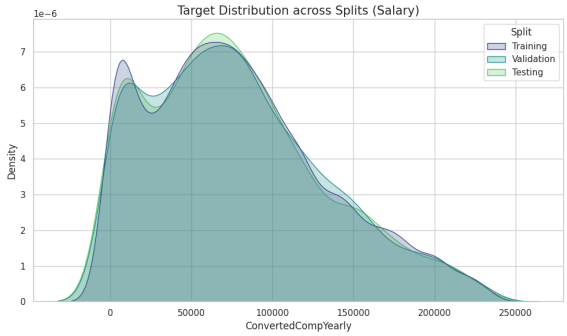


Figure 12: Target Distribution across Splits (Salary). The overlapping curves indicate a statistically representative split.

This consistency is further evidenced by the boxplot analysis in Figure 13, which demonstrates that the Median and Interquartile Ranges (IQR) for salary remain stable across the training, validation, and testing partitions.

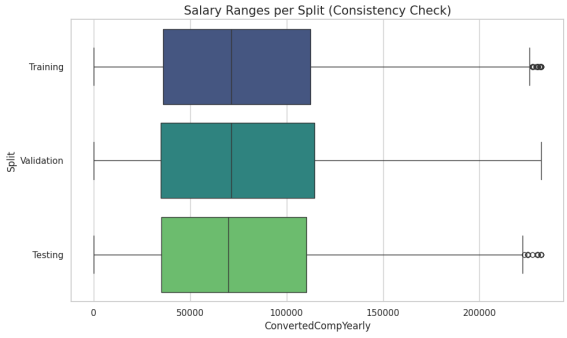


Figure 13: Salary Ranges per Split (Consistency Check).

4.4.2 Feature Distribution Analysis. We extended this verification to key features to ensure the model learns from a balanced demographic. Figure 14 illustrates the normalized distribution of `WorkExp`. The density of experience levels is consistent across splits, ensuring that both junior and senior profiles are equally distributed.

4.5 Training & Tuning Results

Provenance Tracking: The training process was executed as a semantically tracked activity (`:train_and_finetune_model`). To ensure rigorous experiment tracking, every iteration of the grid search was logged as a distinct `mls:Run` entity. Each run links

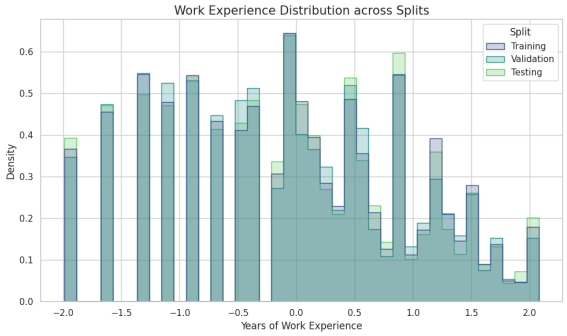


Figure 14: Work Experience Distribution across Splits.

the specific `mls:HyperParameterSetting` (the alpha value) to its resulting `mls:ModelEvaluation` (R^2 and MAE), ensuring that the decision process is fully queryable and reproducible.

4.5.1 Performance Metrics (Hyperparameter Tuning). The model was trained on the 70% training split and evaluated on the 15% validation split across the logarithmic grid $\alpha \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$.

Table 2 details the specific performance metrics for each configuration.

Table 2: Grid Search Results: Regularization Impact

Alpha (α)	R^2 Score	MAE (USD)
0.01	0.5712	26,325.67
0.1	0.5712	26,325.48
1.0	0.5712	26,323.76
10.0	**0.5713**	**26,313.32**
100.0	0.5662	26,500.52

Interpretation:

- **Stability ($\alpha \leq 1.0$):** The model exhibits high stability in the lower range. The R^2 score remains constant at 0.5712, suggesting that mild regularization has little impact on the primary signal.
- **Peak Performance ($\alpha = 10.0$):** The optimal performance is observed at $\alpha = 10.0$. While the R^2 improvement is marginal (+0.0001), the Mean Absolute Error (MAE) decreases by approximately \$10 compared to lower alpha values. This indicates the "sweet spot" where the L2 penalty is strong enough to dampen noise without suppressing relevant features.
- **Degradation ($\alpha = 100.0$):** A sharp drop in performance occurs at $\alpha = 100.0$ (R^2 drops to 0.5662, MAE increases by $\approx$ \$190). Here, the regularization penalty becomes too dominant, forcing coefficients too close to zero and causing the model to underfit.

To confirm the mechanism of this regularization, we visualized the **Coefficient Path** (Figure 16). As α increases (moving right on the log-scale x-axis), the penalty term begins to dominate, forcing the model to reduce the magnitude of the feature weights. When

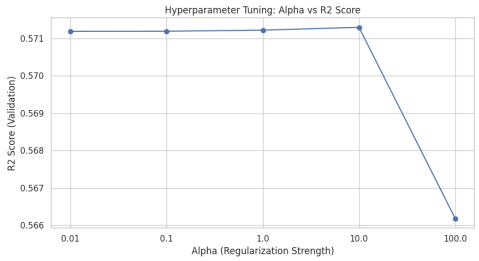


Figure 15: Hyperparameter Tuning Results: Validation R^2 scores across Alpha values.

these coefficients shrink toward zero, the model becomes less sensitive to specific features; in the extreme case, where coefficients approach zero, the model effectively ignores individual feature differences and reverts to predicting the global average salary (underfitting). This rapid convergence of lines, observed most notably at $\alpha = 100.0$, correlates perfectly with the performance drop seen in our tuning results, confirming that at this level the model is "over-dampened" and suppressing valid predictive signals.

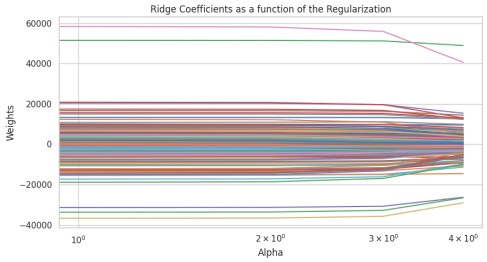


Figure 16: Ridge Coefficient Path: Visualizing the shrinkage of feature weights as regularization strength increases.

4.6 Final Model Selection

Based on the validation results, the hyperparameter configuration with $\alpha = 10.0$ was selected as the optimal model.

Table 3: Final Model Validation Performance

Metric	Value	Interpretation
R^2 Score	0.5713	Explains ~57.1% of variance in salary data.
MAE	$26,313.32	Average prediction error in USD.

An important note is that the metrics reported below represent performance on the **Validation Set (15%)** only. The held-out Test Set remains completely unseen at this stage to ensure it serves as an unbiased evaluator for the final production model.

4.7 Retraining Process

Following the selection of the best hyperparameter ($\alpha = 10.0$), the final production model was not merely the specific artifact from the validation run. Instead, we performed a full **Retraining** step:

(1) **Data Merging:** The Training (70%) and Validation (15%) sets were merged to form a larger training corpus (85% of total data).
(2) **Refitting:** A fresh Ridge Regressor was initialized with $\alpha = 10.0$ and fitted on this merged dataset.
(3) **Justification:** This approach maximizes the data available for the model to learn patterns, likely improving generalization on the unseen Test set (final 15%) without violating the hold-out principle.

4.8 Alternative Approach: Non-Linear Benchmarking (HGB)

To validate the performance of our linear baseline and check for complex non-linear relationships in the salary data, we conducted a comparative experiment using **Histogram-based Gradient Boosting (HGB)**.

4.8.1 Experimental Setup. We evaluated three distinct configurations of the `HistGradientBoostingRegressor`, varying the regularization strength and tree complexity (depth). This exploratory activity was tracked in the provenance graph to distinguish it from the primary production pipeline.

4.8.2 Results & Comparison. The results, summarized in Table 4, demonstrate that the tree-based model slightly outperforms the linear baseline.

Table 4: HGB Benchmarking Results

L2 Regularization	Max Depth	Train R^2	Validation R^2
0.0 (Baseline)	None	0.7093	**0.5957**
1.0 (Moderate)	10	0.7102	0.5955
10.0 (Strong)	5	0.6626	0.5787

Conclusion: The best HGB configuration achieved a validation R^2 of **0.5957**, exceeding the Ridge Regression baseline (0.5713) by approximately 2.4%. This suggests that while linear factors (geography, experience) drive the majority of the prediction, there are non-linear interactions in the data. However, the Ridge model remains the primary selection for this report due to its critical interpretability (e.g., exact dollar-value coefficients for specific countries). However, in further iterations of the CRISP-DM process, the focus should potentially shift towards non-linear models.

5 Evaluation

This chapter now focuses on the performance of the final model, firstly on the performance of the model in isolation and afterwards compared against other models, a relatively low baseline and last but not least against the in the beginning defined success criteria.

5.1 Performance reflection

To evaluate the performance of our model, I calculated the coefficient of determination as well as the mean absolute error. As already partly explained in the data section, our initial results were not satisfactory. Therefore, we performed several iterations of data preparation, modeling, and subsequent evaluation.

In the beginning our R^2 was just about 50%, which was far from satisfaction for us. And after making some changes to the data, our linear model was able to reach a R^2 of nearly 60%.

Since we were still not fully satisfied with this result, additionally a non-linear model has been developed. When trained on the training data, this model achieved an R^2 of up to 70%. Finally, after evaluating both models on the test set, the non-linear model performed slightly better, achieving a R^2 of 60.35% while the linear model achieved a R^2 of 59,15%, which can be considered acceptable. However, its mean absolute error of 25,220.51 is unfortunately still too high.

5.2 State-of-the-art performance

Regarding our dataset, I identified three websites that documented the use of the same dataset as ours. But please note that because our dataset is relatively new, all three models were trained and tested using developer survey data from the previous year. Although this is generally not considered best practice, it was the most feasible approach for us.

The first model was found on Kaggle [2], where only the root mean squared error was reported.

The second model was published on Medium [9]. This model achieved an R^2 of 0.045 and a root mean squared error of 113,000.

The final model was found on GitHub, [12] where three performance metrics were reported: an R^2 of 0.619, a mean absolute error of 25,121.2, and a root mean squared error of 37,068.77.

5.3 Baseline performance

In order to properly calculate a trivial acceptor / baseline performance I calculated the R^2, the MAE and the RMSE:

- R^2 = -0.0005
- MAE = 44.279,54
- RMSE = 54.765,98

For calculating usable baseline performances / trivial acceptors I did some research online and used the baselines stated in [1].

5.4 Comparing to State-of-the-art and baseline

This section now focuses on the performance of our model against the in section 5.2 and 5.3 stated metrics.

5.4.1 Baseline performance. Overall, our model was able to closely match or even beat the reported performance metrics of comparable models found online. Starting with the baseline / trivial acceptor, our model unsurprisingly outperformed all baseline metrics, with none of the baseline results coming close to our model's performance.

5.4.2 State-of-the-art performance. With an RMSE of 33.2k USD on the test set, the Kaggle model performed slightly better than our model, which achieved an RMSE of approximately 34.5k EUR.

After comparing our calculated metrics with those reported in the Medium article, it became evident that the Medium model performs considerably worse. Consequently, our model clearly outperformed all of their reported metrics.

When comparing our model to the GitHub implementation, our model achieved a lower RMSEand performed slightly worse in terms of the R^2 score and the MAE.

In conclusion, our model can be considered close to state-of-the-art performance for this task. None of the compared models significantly outperform our model, and even in cases where marginally better results are achieved, our performance remains very close to the reported benchmarks.

5.4.3 Regression errors. Our model has regression errors in certain parts of the data space, which occur the higher a salary gets. Based on the diagram in 3.2 I subjectively created three salary classes: The lowest regression errors have been recorded within the medium class, while we had more regression error in the Low class and the most errors within the hightest class. The following diagram should help to better visualize this:

Class	Lower Border	Upper Border
Low	0	50 000
Medium	50 000	150 000
High	150 000	max_salary

Table 5: Salary class borders used for regression error analysis

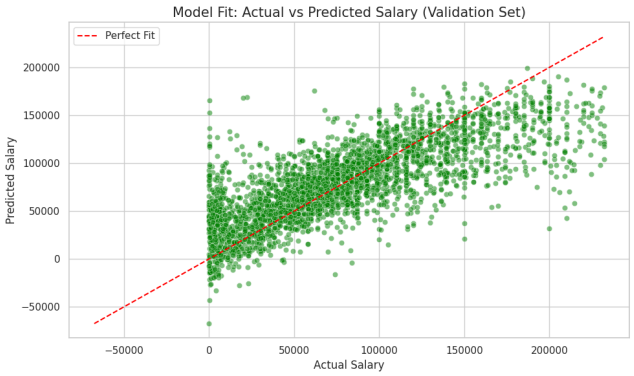


Figure 17: Model Fit: Actual vs Predicted.

5.5 Comparison of the performance against success criteria

This section focuses on the comparison of our model against the data mining success criteria only, as a comparison against the Business success criteria is done later on in the Deployment section. The following are the success criteria to meet:

- R^2 > 80%
- Less than +/- 3000 euro on average
- Less then 10% of missing data
- Readable for HR department

5.5.1 R^2 > 80%. After the modeling of my colleague and my evaluation phase I now know, that the minimum of 80% was very ambitious in the first place, especially after seeing, that other models are reaching worse R^2 values than we do. For future projects we definitely would take the fact that salary data is very noisy into account and therefore would give ourselves a little bit of an easier minimum value to reach.

5.5.2 Less than +/- 3000 euro on average. Unfortunately we also did not meet this goal, as we have a MAE of way past 20.000 euros for both the linear and non-linear model. But even if this now sounds very bad, at the end of the day this goal was also very ambitious, and compared against other models, our model performs quite ok.

5.5.3 Less than 10% of missing data. We did reach this goal, as we got rid of every missing piece of data during the data preparation phase.

5.5.4 Readable for HR department. Personally I think we will never know if we really reached this subjectiv goal, as readability shouldnt be defined be the people that created the model.

5.6 Protected attribute

Regarding our dataset, there there are several protected attributes like the country a developer is working in or the type of working (in-person, remote, hybrid, ...). Here I decided to evaluate if the model exhibits a bias towards the country of a developer. In the following table I calculated the RMSE and MAE for every country separately:

Table 6: Prediction Errors by Country

Country	Linear		Nonlinear	
	RMSE	MAE	RMSE	MAE
USA	42496.38	31857.25	40439.42	30490.53
GER	26837.38	20242.91	27070.88	19757.31
IND	22912.64	17643.28	20079.94	14890.44
UK	35849.82	27721.17	35910.15	28155.25
FRA	27679.68	19872.63	28821.40	20586.54
CAN	32754.75	23870.81	33062.27	23844.00
UKR	30467.90	21878.68	30338.28	21531.61
POL	33092.49	24474.33	34241.52	24891.74
NLD	41334.31	26652.36	42208.58	26785.79
ITA	28196.41	20897.32	28339.53	20664.30
BRA	36228.17	25660.89	33521.21	23107.33
AUS	35333.79	25460.93	36174.63	26880.68
ESP	31027.28	23302.21	31736.44	24350.76
SWE	23127.00	16794.14	25806.90	19094.70
CHE	36611.47	29931.86	33576.73	26887.95
CZE	34890.56	23841.21	34135.53	25188.55
AUT	36088.02	26067.04	38714.88	28172.56
ROU	49382.93	33519.99	45321.30	32121.20
BEL	42169.05	31183.16	42315.31	30746.28
Other	35258.29	26956.90	35291.74	26526.42

Here it is now easy to see, that the model has a bias towards some countries like the United States of America, Romania or Belgium which drive up the RMSE as well as the MAE.

6 Deployment

The final phase of the project focuses on transitioning the developed model from a theoretical solution to a practical tool for Fritz & Maier AG. This section evaluates whether the initial business objectives and success criteria have been met and outlines the recommended hybrid deployment strategy. Furthermore, we address critical ethical considerations regarding fairness and bias, define a comprehensive monitoring plan to handle data drift, and conclude with a reflection on the reproducibility of our analysis using the Knowledge Graph approach.

6.1 Comparison And Recommendations for Business Objectives and Success criteria

This section now compares the at the beginning of the project stated goals and success criterias in order to know, whether we reached our goals. Furthermore, it will be briefly discussed, which type of deployment is preferred for our model.

6.1.1 Business success criteria.

- HR reviewing current employee contracts: This is easily possible because HR can use the model in order immediately see where we have a significant difference to other salaries. This success criteria is therefore met.
- Reducing staff turnover: Even though our model isn't perfect, it still helps standardizing salaries and therefore helps increase the moral of employees which results in less contract terminations. This success criteria is met as well.
- Offer Acceptance: The model helps HR estimate more reasonable and competitive salary offers based on existing data. While the results are not perfect and still need to be reviewed, they can support better initial offers, which can increase the chance that candidates accept the job offer. I'd say this criteria isn't fully met, as HR cannot completely rely on it, but it is quite a good "helper".
- Budget Optimization: While our model helps in that our company neither over- nor underpays, it is unfortunately not perfect, so outliers are easily possible here. We therefore do not know whether we are within +/- 10% of the predicted market benchmark, and so every new employees' salary needs to be checked carefully and validated. Overall, we therefore do not know whether we met this criteria.
- Faster Time-to-Hire: To know whether we reached this goal it will take some time, as we would need an empirical study conducted by the HR department regarding the time-to-hire that was spent. As of today, we cannot clearly tell, whether we reached this success criteria.

6.1.2 Business objectives met.

- Identifying significant salaries of current employees: Thanks to our model, this should be possible now. But regarding the data needed for the job satisfaction it may be hard to get true statements from the employees in a non-anonymous way.
- Standardizing salary offers: As stated within the Business success criteria part, time will tell if we have been able to achieve this criteria, even though we do think we fulfilled this criteria.

- Company budget: As we are now able to better predict what developers will earn, we do achieve this business objective, as the companies CFO is now able to plan the budget with higher reliance.

While we have been able to tell, for some of the business objectives / success criteria, whether we achieved / met these, for others more testing, evaluation and even some refinement of the model will be necessary.

6.1.3 Form of deployment.

- With our model of predicting salaries it would not be wise to fully automate the deployment as well as only deploy it for certain parts of the data. The best thing to do in my opinion would be a hybrid solution, where the model helps in finding a decision, but never does so alone due to bias concerns, etc. There should always be a human in the loop monitoring everything that happens.

6.1.4 Recommendations for subsequent analysis.

- The best way to always be able to analyze how the model predicts a certain salary, is to use logs that give all possible information to the engineers.
- In this regard it would be very important to especially take a look into logs, where our model predicted some nonsense.
- Maybe also take some feedback of HR into account, as they are the main users of our model. This probably will not increase metrics like the models accuracy, but maybe on how to handle it.
- As audits are inevitable for big companies, we as engineers of the model could do internal AI-audits from time to time, where it is checked whether our model works and is used as was aimed for and if we have ways of improving our model. This last part correlates with logging and getting feedback, as a big part of an audit is having evidences and further validating them.

6.2 Ethical Aspects

6.2.1 Ethical aspects in Business Understanding. This section now covers the impact and possible mitigations of the within the Business Understanding and Data Understanding phase stated fairness & ethical risks.

- Gender equality: As already stated, our dataset does not differentiate between different genders, which therefore lowers a risk of a bias in that regard.
- Noisy & missing data: During our data preparation phase we got rid of every piece of missing data. Therefore the risk of having missing data is low. However the dataset still contains noisy data which could influence our model in a negative way. This could impact the decision making in a way, that false conclusions are drawn.
- Country of a developer: This unfortunately created a bias within our system and the best way to mitigate the impact would probably be to remove the country. Unfortunately this is not viable possibility, as there are in taxation alone a lot of differences that impact the salary prediction. Not taking this feature into account could lead to drawing false conclusions as well.
- Population of developers: This still is a big risk, as we did not take an equal amount of samples to train our models, which could lead to the under representation of certain groups and consequently lead to worse / unfair predictions.
- Fritz & Maier AG salaries: Obviously there always is a risk that a created model does not or not sufficient enough do what we want it to do. But even we do have a unique way of setting salaries right now, we want to change that for the future, and therefore the risk of this can be considered as low.
- Governance & documentation: We are doing our best to mitigate risks here, as I already proposed the usage of logging, feedback from HR and internal audits.

6.2.2 Ethical aspects in Data Understanding.

- Dataset suitable for predicting salaries: As stated within the data understanding part we do not know whether our dataset is suitable for predicting an individual's salary. But an extensive survey like that is our best chance to get a large amount of real world data with little or no money invested. So, in order not to have the unfair predictions regarding individuals, every predicted salary should be checked by HR. If not done, this could have a big negative impact on our time-to-hire, because people for sure won't join the company, if the initial salary proposal strays far from what would actually be suitable and further even if they joined the company, this employee would not be happy and therefore probably don't stay for long.
- Unfair treatment: Generally it could always happen, that certain groups are getting an unfair treatment, this always could have a negative impact. We are now getting into detail about the following stated examples that were mentioned within the Data Understanding part: Country origins, the age of a developer or his or her learning path. For the country of a developer, the impact of this risk as well as possible mitigations have already been discussed in sections 5.6 and 6.2.1. Regarding ageism, this could have a negative impact in predicting salaries, and therefore in order to mitigate this problem, HR has to carefully review every predicted salary. Last but not least, wrong predictions due to the educational level of a developer can also have a big negative impact. In this regard, the probably easiest way to mitigate this, is to have in addition to HR the possible future supervisor of this applicant check, whether the predicted salary is in accordance with the skill level.

6.3 Monitoring Plan

To ensure the model maintains its validity in a production environment, we have established a comprehensive monitoring strategy focusing on data drift, performance tracking, and ethical fairness.

6.3.1 Data Drift & Concept Drift. Salary data is intrinsically temporal. A model trained on 2025 data will likely underestimate salaries in 2026 due to inflation. To mitigate this, we propose a quarterly "Index Adjustment" where model predictions are validated against current market rates (using fresh scraped data or CPI indices) to

detect if the baseline is drifting. Furthermore, the technology landscape evolves rapidly. We must continuously monitor the input distribution of the "Skills" and "DevType" features; if the frequency of a new skill (e.g., "Mojo" or "Q#") spikes in new applications but is absent or rare in our training distribution, the model will be flagged for retraining.

6.3.2 Performance Monitoring (Human-in-the-Loop). As this model is designed as a decision-support tool rather than an automated agent, we will implement a "Prediction vs. Actual" log. HR personnel will record the final signed offer alongside the model's initial prediction. If the Mean Absolute Error (MAE) of the deployed model exceeds the Test Set MAE by more than 10% over a rolling 3-month period, a retraining alert is triggered. Additionally, we will actively monitor the rate of "Outlier Flags" generated by the deployment pipeline. If the model flags more than 20% of new applicants as outliers (indicating salary expectations are systematically too high or low), it suggests the underlying data distribution has shifted significantly compared to the 2025 survey baseline.

6.3.3 Bias Auditing. Finally, to ensure ethical fairness, we will execute the bias evaluation script (established in the Evaluation phase) on newly accumulated applicant data annually. This ensures the model does not develop or amplify bias against specific geographic regions or non-traditional educational backgrounds over time.

6.4 Reflection on Reproducibility

6.4.1 Strengths via Provenance & Environment. The project exhibits robust reproducibility through several mechanisms. First, by logging every activity (e.g., `:handle_outliers`, `:scale_columns`) and entity into the Knowledge Graph, we have created a semantic trail. This allows auditors to trace exactly which transformation steps led to the final model, exceeding standard code documentation by making the process machine-queryable. Second, we ensured determinism by utilizing `random_state=42` for all stochastic processes, ensuring that re-running the notebook yields the exact same coefficients and R^2 scores given the same input. Finally, the use of a dedicated Conda environment (defined via `requirements.txt`) ensures that library version discrepancies—such as Scikit-Learn updates altering default solver behaviors—do not alter the results.

6.4.2 Limitations & Challenges. Despite these measures, certain limitations persist. While the code execution is reproducible, the *rationale* behind specific manual decisions—such as the choice of outlier thresholds or the exclusion of columns like `RemoteWork`—remains subjective. While the graph documents *what* happened, reproducing the *thought process* requires this accompanying report. Additionally, the project relies on the external Stack Overflow 2025 CSV file; if this source file is modified or removed from the data directory, the graph pointers will break. True reproducibility would require version-controlling the raw data artifact (e.g., using DVC) alongside the code. Finally, execution times logged in the graph are hardware-dependent, meaning that efficiency metrics cannot be guaranteed across different machines.

7 Conclusion

In this project, we successfully applied the CRISP-DM methodology to the Stack Overflow 2025 Annual Developer Survey to develop a salary benchmarking tool for Fritz & Maier AG. The primary business objective was to establish a data-driven model capable of identifying internal pay inequities and standardizing offers for new hires.

Through iterative cycles of data understanding and preparation, we developed a linear baseline model using **Ridge Regression**, which achieved an R^2 of 0.5713 and a Mean Absolute Error (MAE) of $26,313 on the validation set. To validate these results against non-linear complexities, we trained a comparative **Histogram-based Gradient Boosting (HGB)** model, which improved performance to a Test R^2 of 60.35%.

Our analysis revealed several key insights:

- **Model Performance:** Our model outperformed the trivial baseline and achieved results comparable to state-of-the-art benchmarks found on Kaggle and GitHub. However, precise salary prediction remains challenging due to the extreme noise and right-skewness of the data.
- **Regression Errors:** We observed that regression errors increase with higher salary brackets. While the model performs adequately for low-to-medium salary ranges, it struggles with the highest bracket (> $150,000).
- **Business Viability:** Regarding the business objectives, the model is best deployed as a helper tool rather than an autonomous agent. It successfully meets the criteria for reviewing current contracts and reducing turnover risk but requires human supervision for budget optimization due to potential outliers.

7.1 Lessons Learned

Throughout the lifecycle of this project, we encountered several challenges that provided valuable learning opportunities regarding the data mining process.

A significant methodological lesson involved the dependency between *Data Understanding* and *Data Preparation*. We found that certain exploratory analyses were only meaningful *after* specific preprocessing steps had been applied. However, strictly adhering to the initial project plan sometimes caused friction, as we had to partially transform data before fully analyzing it. This reinforced the non-linear, cyclic nature of CRISP-DM, where moving back and forth between phases was necessary to derive valid insights.

Additionally, we learned that standard statistical techniques must be adapted to the domain. For instance, using the standard Interquartile Range (IQR) method for outlier removal proved destructive, as it flagged experienced professionals (40+ years of tenure) as outliers. Adopting a log-transformation combined with percentile capping allowed us to retain these valuable signals while stabilizing the model.

7.2 Feedback on the Exercise

Reflecting on the course structure and the assignment, we would like to offer the following feedback:

- **Project Topic:** Overall, we found the project topic to be quite interesting and engaging. The real-world applicability of the Stack Overflow dataset made the workload feel relevant and manageable.
  item **Process and Instructions:** We noticed discrepancies

between the specific assignment instructions and the standard CRISP-DM process definition. At times, this divergence led to confusion regarding the expected deliverables for specific phases. Clearer alignment would help future students navigate the workflow.

- **Documentation Workflow:** We found the workflow of generating the report directly from Python comments to be problematic. Since code comments are written in a specific technical style (often using special symbols for structure), they are rarely suitable for direct inclusion in a formal report. This necessitated significant rewriting and manual editing of the retrieved text, effectively doubling the documentation effort and leading to inevitable inconsistencies between the content stored in the Provenance Graph and the final submitted report.
- **Provenance Graph:** Regarding the documentation via the Provenance Graph, we faced significant challenges. While we understand that bugs and server downtime are sometimes unavoidable, these technical issues consumed a disproportionate amount of time. Since the focus of the course is data mining rather than graph database interaction, we suggest providing a small **SDK** or a Python wrapper for future iterations. This would abstract away the raw graph interaction, allowing students to focus on the content of the documentation.
- **Course Content:** Finally, we believe the curriculum would benefit from a dedicated section on **Regression Analysis**. While classification is critical, regression is a fundamental aspect of data mining that would have provided a more well-rounded skillset for this specific project.

8 Divergence from Provenance Documentation

This report complements the accompanying Provenance Knowledge Graph. The graph serves as our technical 'ground truth'. acting as an exact registry for every experimental detail, including execution timestamps, raw statistics, and algorithm parameters. In contrast, this document focuses on narrative and interpretation. To ensure coherence and readability, we have synthesized the raw logs from the graph and slightly reworked the descriptions into a structured academic format. While the Knowledge Graph contains the precise data required for reproducibility, this report expands upon those facts to explain the context and reasoning behind our decisions.

9 AI-Disclaimer

As discussed in the lecture with Professor Rauber, we used Generative AI as a helper throughout this project to support several different tasks, beginning with the brainstorming phase and the clarification of specific assignment requirements. It provided structured feedback on our adherence to the CRISP-DM process and served as a knowledge base for exploring fundamental concepts within Regression Machine Learning. For the technical part AI helped us choose the right algorithms, fix bugs in our code, and log our steps into the knowledge graph. Finally, we used it to improve the wording and make the text clearer in both this report and the Provenance Ontology documentation.

References

[1] DagsHub Glossary. Baseline models. https://dagshub.com/glossary/baseline-models/, 2026. Accessed: 2026-01-13.

[2] dima806. Stackoverflow Survey 2024 Salary ML+SHAP. https://www.kaggle.com/code/dima806/stackoverflow-survey-2024-salary-ml-shap, 2024. Kaggle Notebook, accessed 16 Jan. 2026.

[3] GeeksforGeeks. Yeo-johnson transform, 2025. URL https://www.geeksforgeeks.org/machine-learning/yeo-johnson-transform/. Last updated 23 Jul 2025.

[4] Trevor Hastie. The elements of statistical learning: data mining, inference, and prediction, 2009.

[5] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.

[6] Qing Li and Nan Lin. The bayesian elastic net. 2010.

[7] Gary C McDonald. Ridge regression. *Wiley Interdisciplinary Reviews: Computational Statistics*, 1(1):93–100, 2009.

[8] Jonas Ranstam and Jonathan A Cook. Lasso regression. *Journal of British Surgery*, 105(10):1348–1348, 2018.

[9] Pratiti Soumya. Predicting developer salaries with machine learning, June 2025. URL https://medium.com/@pratitisoumya/predicting-developer-salaries-with-machine-learning-f2d81579af86. Medium blog post.

[10] Stack Overflow. 2025 annual developer survey, 2025. URL https://survey.stackoverflow.co/2025/. Accessed: 2026-01-03.

[11] The scikit-learn developers. sklearn.linear_model.ridge — scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html, 2024. Accessed: 2026-01-05.

[12] tien02. salary-prediction: Predicting developer's salary from Stack Overflow Annual Developer Survey. https://github.com/tien02/salary-prediction, 2026. GitHub repository, accessed 16 Jan. 2026.